MANUAL TÉCNICO
FINANCE AI

Registro y análisis de finanzas personales con asistencia de IA

Integrantes:
David Fernando Gonzalez Soler
Maycol Estiven Fonseca Muñoz
Ivan David Giraldo Garavito

Programa: Ingeniería de Software
Universidad de Cundinamarca
Noviembre de 2026

TABLA DE CONTENIDO

1 INTRODUCCIÓN **6**

2 OBJETIVO **6**

3 GENERALIDADES **6**

3.1 DESCRIPCIÓN DEL SISTEMA 6

3.2 REPOSITORIOS 7

3.3 ARQUITECTURA 7

3.4 MÓDULOS DEL SERVIDOR 8

3.5 MÓDULOS DEL CLIENTE 8

3.6 API REST 9

3.7 SEGURIDAD Y PRIVACIDAD 9

3.8 FLUJOS PRINCIPALES 10

3.9 DESPLIEGUE Y ENTORNOS 12

3.10 CALIDAD Y PRUEBAS 13

4 PRERREQUISITOS DEL SISTEMA **13**

4.1 HARDWARE 13

4.2 SOFTWARE 13

4.3 INSTALACIÓN Y PUESTA EN MARCHA 14

5 FRAMEWORK Y ESTÁNDARES **14**

5.1 FRAMEWORKS Y LIBRERÍAS 14

5.2 ESTÁNDARES 15

6 DIAGRAMAS DE CASOS DE USO **16**

6.1 REGISTRO E INICIO DE SESIÓN 17

6.2 CUENTAS DE DINERO 17

6.3 REGISTRO DE MOVIMIENTOS 18

6.4 CORRECCIÓN DE GASTOS E INGRESOS 18

6.5 HISTORIAL Y PANEL FINANCIERO 19

6.6 CATEGORÍAS 19

6.7 FUENTES DE INGRESO 20

6.8 PRESUPUESTOS 20

7 MODELO ENTIDAD RELACIÓN **21**

7.1 IDENTIDAD Y SESIONES 22

7.2 CUENTAS Y TRANSACCIONES 23

7.3 CATÁLOGOS Y RECIBOS 24

7.4 GASTOS 25
7.5 PRESUPUESTOS Y CAPTURAS CON IA 26
8 DICCIONARIO DE DATOS DEL SISTEMA 26
8.1 IDENTIDAD Y SESIONES 27
8.1.1 Tabla role 27
8.1.2 Tabla app_user 27
8.1.3 Tabla user_session 27
8.1.4 Tabla renewable_credential 28
8.2 REFERENCIA 28
8.2.1 Tabla currency 28
8.3 CUENTAS Y TRANSACCIONES 28
8.3.1 Tabla account 28
8.3.2 Tabla financial_transaction 29
8.3.3 Tabla account_movement 29
8.4 CATÁLOGOS 30
8.4.1 Tabla category 30
8.4.2 Tabla subcategory 30
8.4.3 Tabla paymentmethod 30
8.4.4 Tabla income_source 31
8.4.5 Tabla incomesource 31
8.4.6 Tabla store 31
8.5 RECIBOS, GASTOS E INGRESOS 31
8.5.1 Tabla receipt 31
8.5.2 Tabla expense 32
8.5.3 Tabla income 33
8.6 PRESUPUESTOS 33
8.6.1 Tabla budget 33
8.6.2 Tabla spendinglimit 34
8.7 CAPTURAS CON IA 34
8.7.1 Tabla receipt_proposals 34
8.7.2 Tabla text_capture_proposals 34
9 ANEXOS 35
9.1 PROCEDIMIENTOS ALMACENADOS Y DISPARADORES 35
9.1.1 Mecanismos de la base de datos equivalentes a disparadores 35
9.1.2 Operaciones atómicas del servidor equivalentes a procedimientos almacenados 35
9.2 EVIDENCIAS DE EJECUCIÓN 36

9.2.1 Migraciones de la base de datos 36
9.2.2 Servicios en ejecución y comprobación de salud 36
9.2.3 Funcionamiento del cliente 37

REGISTRO DE CAMBIOS		
VERSIÓN	**FECHA DE CAMBIO**	**MOTIVO DEL CAMBIO**
1.0	13/11/2026	Elaboración inicial

CONTROL DEL DOCUMENTO			
VERSIÓN	**ELABORADO**	**REVISADO**	**APROBADO**
1.0	David Fernando Gonzalez Soler Maycol Estiven Fonseca Muñoz Ivan David Giraldo Garavito	N/A	Oscar Gómez

1 INTRODUCCIÓN

Finance AI es una aplicación para registrar y analizar finanzas personales. La persona usuaria administra sus cuentas, categorías y presupuestos, registra gastos, ingresos y transferencias de forma manual o describiéndolos con sus propias palabras (texto, voz o foto de un recibo) y consulta su historial y un panel de resumen. La inteligencia artificial solo propone: ningún movimiento se guarda hasta que la persona lo confirma.

El sistema se compone de un cliente móvil y de escritorio (Flutter, para Android y Windows), un servidor de aplicaciones (FastAPI sobre Python 3.14) y una base de datos PostgreSQL 18.6. Este manual describe la versión en línea del sistema: el cliente necesita conexión con el servidor para operar.

El documento explica cómo está construido el sistema, qué requisitos exige, qué tecnologías y estándares emplea y cómo se organizan sus datos. Se complementa con el *Manual de usuario* (uso de la aplicación) y con el *Diccionario de datos* (detalle de tablas y columnas).

2 OBJETIVO

Proporcionar a desarrolladores, evaluadores y personal de operación la información técnica necesaria para comprender, instalar, configurar, ejecutar, verificar y mantener Finance AI: arquitectura, componentes, requisitos, tecnologías, estándares, seguridad, casos de uso, modelo de datos y evidencias de ejecución.

3 GENERALIDADES

3.1 DESCRIPCIÓN DEL SISTEMA

Finance AI sirve para llevar el control de las finanzas personales. Incluye un registro manual completo y tres modos de captura asistida por IA: texto, voz y foto de un recibo. Los datos financieros residen en el servidor; el cliente solo conserva en memoria el token de acceso y, en el almacén seguro del dispositivo, la credencial renovable.

Tabla 1. *Resumen del sistema.*

Característica	Detalle
Versión del sistema	Servidor `0.1.0` (API) · cliente `0.8.0+8`
Plataformas del cliente	Android (7 o superior) y Windows, con el mismo código Dart
Servidor	API REST con FastAPI, Python 3.14, SQLAlchemy 2.0 y Alembic
Base de datos	PostgreSQL 18.6 (imagen `postgres:18.6-bookworm`)
Contrato de la API	OpenAPI 3.1 generado por el servidor (`openapi.json`), versión de rutas `/api/v1`
Idioma de la interfaz	Español
Rama entregada	`main` de los repositorios del servidor y del cliente

3.2 REPOSITORIOS

Tabla 2. *Repositorios del proyecto.*

Repositorio	Contenido	Dirección
`finance-ai-backend`	API FastAPI, modelos, migraciones, semillas, pruebas y contrato `openapi.json`	https://github.com/Maicol2363/finance-ai-backend
`finance-ai-frontend`	Cliente Flutter compartido para Android y Windows, pruebas y runners nativos	https://github.com/Maicol2363/finance-ai-frontend
`finance-ai`	Documentación del proyecto, diagramas, plantillas y entregables (`docs/entrega`)	https://github.com/NullCorvus/finance-ai

3.3 ARQUITECTURA

El sistema sigue una arquitectura cliente-servidor en tres capas. El cliente Flutter presenta la interfaz y se comunica con el servidor mediante HTTP y JSON. El servidor valida, aplica las reglas de negocio y es el único que accede a la base de datos y a los proveedores de IA. La base de datos PostgreSQL vive en un contenedor privado que no se expone fuera de la red de Docker Compose.

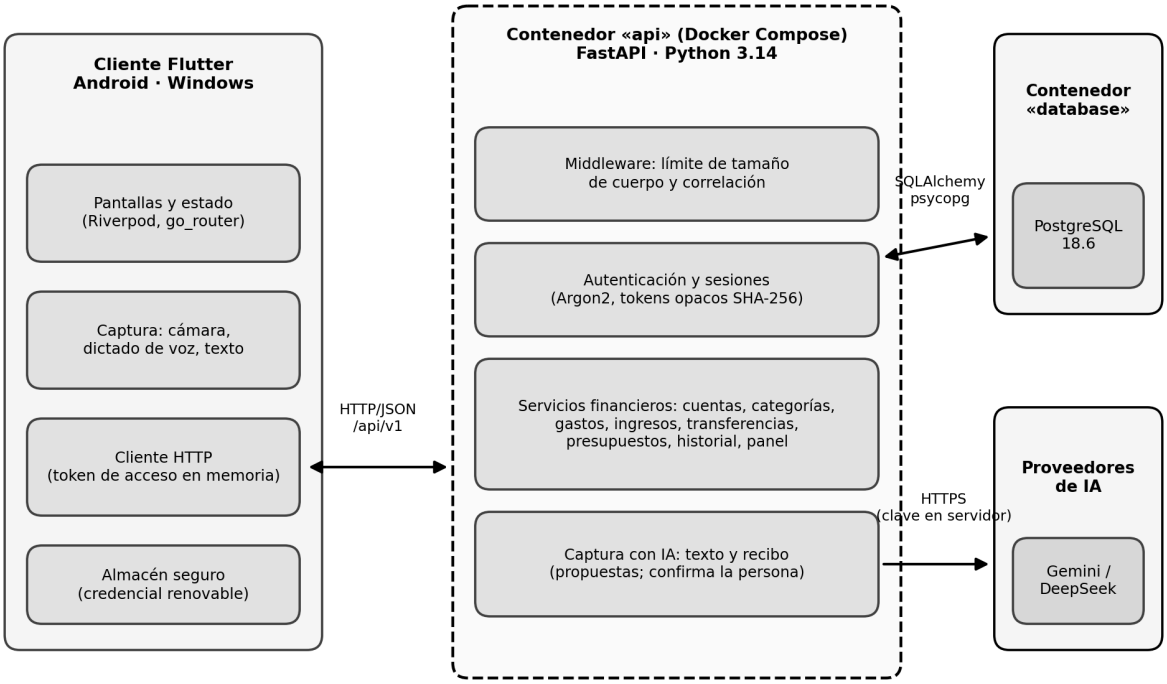


Figura 1. *Arquitectura general de Finance AI.*

El cliente tiene pantallas con Riverpod para el estado y `go_router` para la navegación, un cliente HTTP que mantiene el token de acceso en memoria, captura por cámara, dictado de voz y texto, y un almacén seguro para la credencial renovable.

El servidor es una aplicación modular. Un *middleware* aplica el límite de tamaño de cuerpo y el identificador de correlación; los límites de tasa se aplican en la autenticación; los routers delegan en servicios que usan repositorios y unidades de trabajo transaccionales.

La captura con IA usa pasarelas a Gemini (por defecto) o DeepSeek, con respaldo si el proveedor elegido falla. Las claves residen solo en el servidor. El estado financiero se guarda en PostgreSQL y el saldo de cada cuenta se calcula, nunca se almacena.

3.4 MÓDULOS DEL SERVIDOR

Tabla 3. *Módulos del servidor.*

Paquete (`src/finance_ai`)	**Responsabilidad**
`api`	Router raíz que agrupa los routers versionados bajo `/api/v1`.
`auth`	Registro, inicio y cierre de sesión, renovación de credenciales, hash de contraseñas (Argon2) y límites de tasa.
`accounts`	Cuentas de dinero: alta, edición, desactivación, borrado seguro y saldos derivados.
`categories`	Categorías de gasto con color, ícono y orden.
`income_sources`	Fuentes de ingreso de cada persona.
`expenses`	Gastos manuales y corrección de gastos confirmados.
`incomes`	Ingresos manuales, edición y eliminación.
`transfers`	Transferencias entre cuentas propias, con soporte de monedas distintas.
`budgets`	Presupuestos con calendarios recurrentes o de una sola vez y avance derivado.
`dashboard`	Resumen financiero calculado con SQL (ingresos, gastos, balance y análisis).
`receipts`	Captura de recibos con IA: pasarelas, sanitización de la respuesta y borradores.
`capture`	Captura por texto: interpretación, aclaraciones, confirmación idempotente.
`core`	Configuración, errores estables, middleware, registro de eventos, observabilidad y límites de tasa.
`db`	Modelos SQLAlchemy, repositorios, unidades de trabajo, sesión y semillas.

3.5 MÓDULOS DEL CLIENTE

Tabla 4. *Módulos del cliente.*

Carpeta (`lib/features`)	**Funcionalidad**
`auth`	Registro, inicio de sesión, restauración de la sesión y cierre de sesión.
`home`, `dashboard`	Pantalla Inicio con saldo, presupuestos y análisis de gastos.
`accounts`, `categories`, `income_sources`	Gestión de cuentas, categorías y fuentes de ingreso.
`expenses`, `incomes`, `transfers`	Formularios de registro, detalle, edición e historial de movimientos.
`budgets`	Lista y formulario de presupuestos.
`text_capture`, `voice_capture`	Chat de captura con IA y dictado de voz (reconocimiento local en el dispositivo).
`receipt_capture`	Cámara, comprobación previa, revisión y confirmación de recibos.
`settings`	Ajustes, tema claro u oscuro y cierre de sesión.

Carpeta (`lib/features`)	Funcionalidad
`core`	Navegación, tema, componentes compartidos y diagnósticos copiables de errores.

3.6 API REST

El servidor expone 32 rutas bajo `/api/v1` más dos comprobaciones de salud. Todas las rutas, salvo las de registro e inicio de sesión, exigen el token de acceso en el encabezado `Authorization`. La propiedad de los datos se deriva de la sesión, nunca de un identificador enviado por el cliente.

Tabla 5. *Grupos de la API REST.*

Grupo	Rutas principales	Métodos
Autenticación	`/auth/register`, `/auth/login`, `/auth/refresh`, `/auth/me`, `/auth/logout`	POST, GET
Cuentas	`/accounts`, `/accounts/{id}`, `/accounts/{id}/status`	GET, POST, PATCH, DELETE
Categorías	`/categories`, `/categories/{id}`, `/categories/order`	GET, POST, PATCH, PUT, DELETE
Fuentes de ingreso	`/income-sources`, `/income-sources/{id}`	GET, POST, PATCH, DELETE
Gastos	`/expenses`, `/expenses/{id}`	GET, POST, PATCH, DELETE
Ingresos	`/incomes`, `/incomes/{id}`	GET, POST, PUT, DELETE
Transferencias	`/transfers`	POST
Presupuestos	`/budgets`, `/budgets/{id}`	GET, POST, PATCH, DELETE
Historial	`/history` (búsqueda, filtros y paginación)	GET
Panel	`/dashboard/summary`	GET
Captura por texto	`/capture/text/proposals`, …/`{id}`, …/`{id}`/`confirm`	POST, GET, DELETE
Captura de recibos	`/receipts/proposal`, …/`{id}`, …/`{id}`/`confirm`, …/`{id}`/`retry`, `/receipts/scan`	POST, GET, DELETE
Salud	`/health/live`, `/health/ready`	GET

Los errores usan un formato estable: un código legible por máquina, un mensaje seguro y el identificador de correlación (`correlation_id`), que también viaja en el encabezado `X-Correlation-ID`. Los mensajes nunca incluyen contraseñas, tokens, valores de formularios ni detalles internos de la base de datos.

3.7 SEGURIDAD Y PRIVACIDAD

Tabla 6. *Medidas de seguridad y privacidad.*

Aspecto	Medida
Contraseñas	Se guardan como hash Argon2; nunca en claro.

Aspecto	Medida
Sesiones	Tokens opacos. En la base solo se guarda su hash SHA-256 (64 caracteres hexadecimales).
Vigencia	Token de acceso de 15 minutos; credencial renovable con 30 días de inactividad y 90 días absolutos; máximo 10 sesiones activas por persona.
Renovación	Cada renovación consume la credencial vigente y emite otra (rotación); una credencial ya consumida se detecta como reutilización (*replay*) y se rechaza.
Límites de tasa	Por defecto: registro 10/min, inicio de sesión 10/min (5/min por cuenta) y renovación 30/min por cliente, con topes globales configurables.
Tamaño de peticiones	Cuerpo JSON limitado a 16 KiB; subida de recibos hasta 10 MiB más 64 KiB.
Aislamiento de datos	Toda consulta y mutación filtra por el propietario de la sesión; un identificador ajeno responde igual que uno inexistente.
IA	Las claves de Gemini y DeepSeek viven en variables de entorno del servidor; el cliente nunca las recibe.
Fotos de recibos	Se procesan en memoria; el servidor no conserva el archivo, solo la propuesta estructurada.
Cliente	Token de acceso solo en memoria; credencial renovable en almacenamiento seguro nativo; copias de seguridad de Android desactivadas.
Diagnósticos	Los errores copiables incluyen solo código, operación, hora UTC y correlación; excluyen correo, contraseñas y montos.

3.8 FLUJOS PRINCIPALES

La figura siguiente muestra la secuencia de inicio de sesión, con la emisión y la rotación de credenciales.

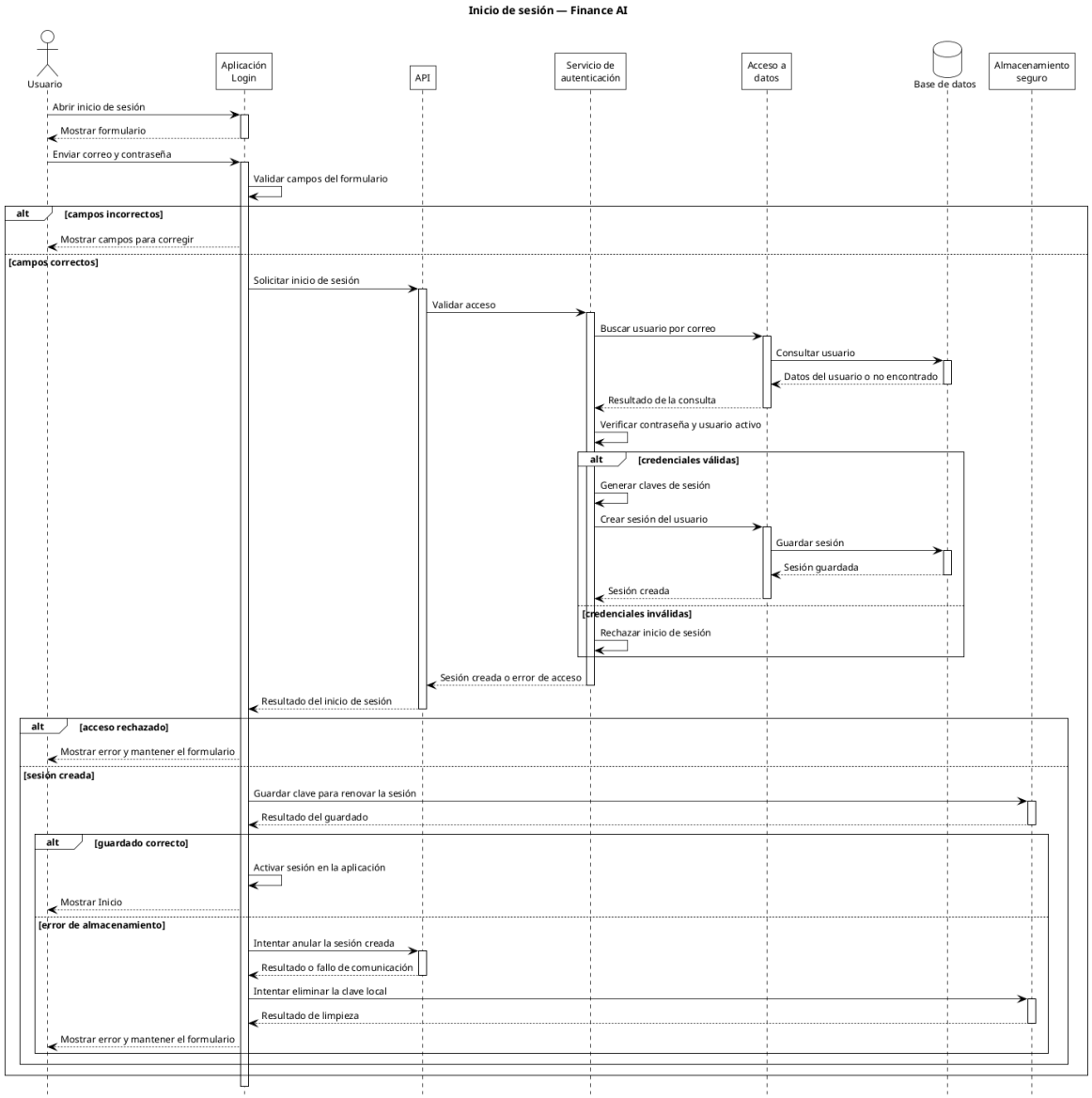


Figura 2. *Diagrama de secuencia del inicio de sesión.*

La captura de un recibo con IA recorre el cliente, el servidor y el proveedor; la persona confirma o descarta la propuesta.

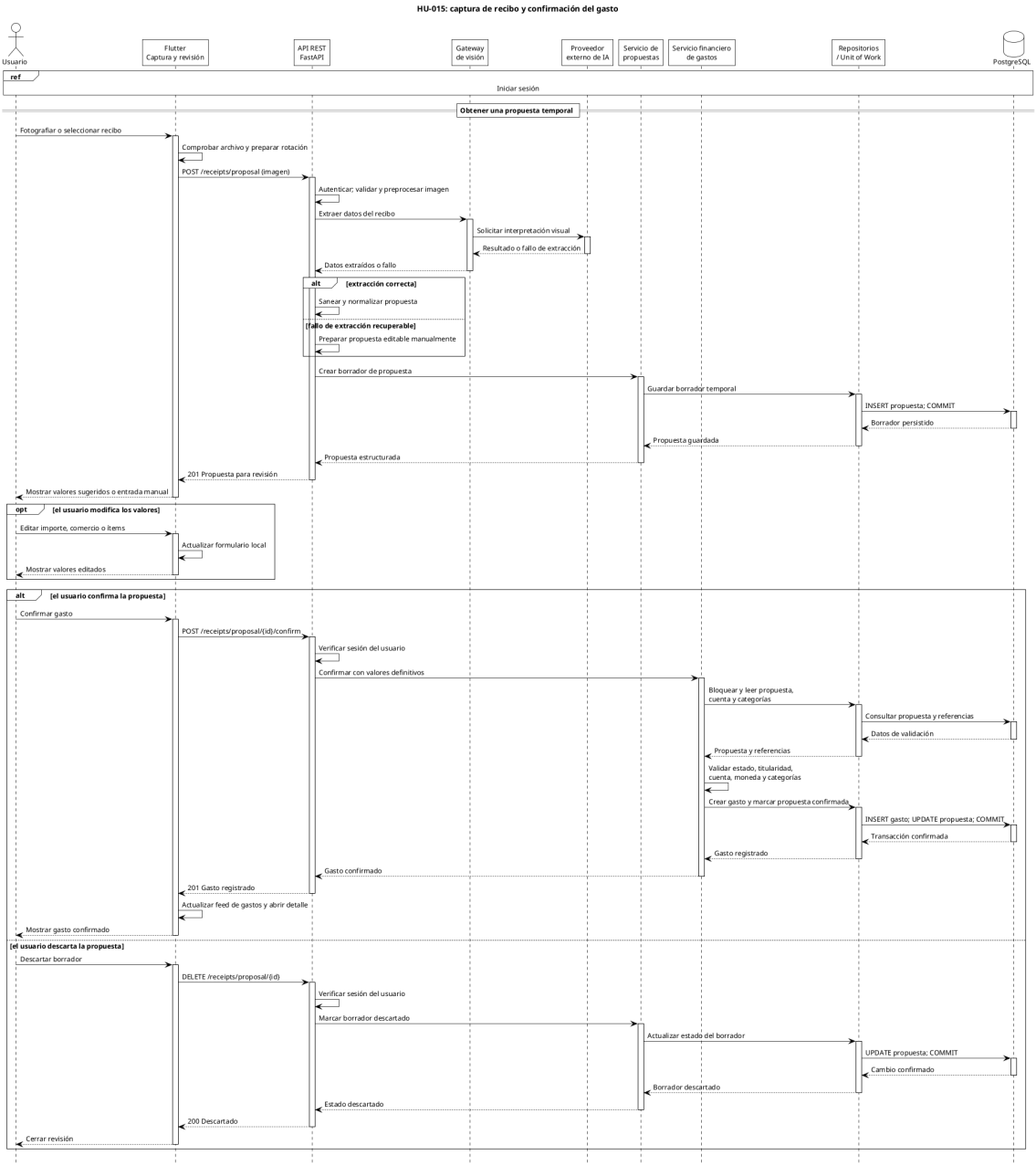


Figura 3. *Diagrama de secuencia de la captura de un recibo.*

3.9 DESPLIEGUE Y ENTORNOS

Para desarrollo local, `docker compose up --build -d` levanta dos contenedores: `database` (PostgreSQL 18.6, con comprobación de salud) y `api` (FastAPI). La base de datos solo es accesible desde la red privada de Compose. El repositorio del servidor define también un flujo de integración continua (*Backend CI*) y un flujo de despliegue a Google Cloud Run que solo se activa para la rama `main`. La existencia de ese flujo no equivale a un despliegue vigente.

3.10 CALIDAD Y PRUEBAS

Tabla 7. *Estrategia de pruebas.*

Tipo	Herramienta	Alcance
Unitarias	pytest	29 archivos: servicios, hashing, límites de tasa, sanitización de recibos, configuración.
API	pytest + httpx	14 archivos: contratos de cada grupo de rutas y casos de error.
Integración	pytest + PostgreSQL	10 archivos: migraciones, persistencia y concurrencia sobre una base `_test`.
Propiedades	Hypothesis	5 archivos: invariantes de montos, nombres y registro.
Estilo y tipos	Ruff, mypy	Formato, lint y tipado estático del código fuente.
Dependencias	pip-audit	Auditoría de vulnerabilidades en las dependencias.
Cliente	flutter_test, integration_test	Pruebas de widgets e integración; cobertura mínima del 80 %.

4 PRERREQUISITOS DEL SISTEMA

4.1 HARDWARE

Tabla 8. *Hardware requerido.*

Elemento	Requisito
Equipo de desarrollo y servidor local	Computador de 64 bits con virtualización habilitada (necesaria para Docker Desktop con WSL 2 en Windows) y espacio libre suficiente para las imágenes de Docker, el SDK de Flutter y el SDK de Android.
Cliente Android	Teléfono con Android 7 o superior, con cámara y micrófono para la captura por foto y por voz.
Cliente Windows	Equipo con Windows 10 u 11 de 64 bits.
Red	Conexión entre el cliente y el servidor: misma red local, USB con `adb reverse` o internet, según el despliegue.

4.2 SOFTWARE

Tabla 9. *Software requerido.*

Nombre	Versión	Descripción
Python	3.14	Lenguaje del servidor.
uv	0.12 o superior	Gestor de entornos y dependencias del servidor (`uv sync --frozen`).
Docker Desktop y Compose	Engine 29 o superior	Contenedores de la API y de PostgreSQL.
PostgreSQL	18.6	Base de datos relacional (imagen `postgres:18.6-bookworm`).
Flutter / Dart	3.47.2 / 3.13.2	SDK del cliente para Android y Windows.
Android SDK y `adb`	Plataforma 7 o superior	Compilación e instalación en teléfonos y emuladores.
Visual Studio (carga de trabajo C++)	2022 o superior	Compilación nativa del cliente de Windows.
Git y GitHub CLI	Reciente	Control de versiones y flujo de ramas.

4.3 INSTALACIÓN Y PUESTA EN MARCHA

Los siguientes pasos levantan el sistema completo en un equipo de desarrollo:

1. Clonar los repositorios `finance-ai-backend` y `finance-ai-frontend` (rama `main`).
2. En el servidor, copiar la plantilla de variables: `Copy-Item .env.example .env` (PowerShell) o `cp .env.example .env`. No se debe subir un `.env` real al repositorio.
3. Iniciar los contenedores: `docker compose up --build -d`. La API aplica las migraciones al arrancar.
4. Comprobar la salud: `GET http://127.0.0.1:8000/health/ready` debe responder `{"status":"ready"}`.
5. Para usar la captura con IA, definir `FINANCE_AI_LLM_PROVIDER` y la clave del proveedor (`FINANCE_AI_GEMINI_API_KEY` o `FINANCE_AI_DEEPSEEK_API_KEY`) en el `.env` y reiniciar la API. Sin clave, la captura con IA responde `text_capture_not_configured` y la aplicación ofrece el formulario manual.
6. En el cliente: `flutter run -d windows` o `flutter run` con un teléfono conectado. La dirección del servidor se cambia con `--dart-define=FINANCE_AI_API_BASE_URL=http://host:8000` (por defecto `http://127.0.0.1:8000` en Windows y `http://10.0.2.2:8000` en el emulador de Android).
7. Para un teléfono físico por USB: `adb reverse tcp:8000 tcp:8000`.

Comandos de verificación del servidor:

Tabla 10. *Comandos de desarrollo del servidor.*

Propósito	Comando
Instalar dependencias	`uv sync --all-groups --frozen`
Aplicar migraciones	`uv run alembic upgrade head`
Cargar semillas	`uv run python -m finance_ai.db.seeds`
Pruebas con cobertura	`uv run pytest --cov=finance_ai --cov-report=term-missing`
Pruebas de integración aisladas	`docker compose --profile test run --rm test`
Estilo, formato y tipos	`uv run ruff check .` · `uv run ruff format --check .` · `uv run mypy src`

5 FRAMEWORK Y ESTÁNDARES

5.1 FRAMEWORKS Y LIBRERÍAS

Servidor (Python):

Tabla 11. *Dependencias de ejecución del servidor.*

Nombre	Versión	Descripción
FastAPI	>=0.140, <0.141	Framework web de la API REST y generación del contrato OpenAPI.
Uvicorn	>=0.38, <1	Servidor ASGI.

Nombre	Versión	Descripción
SQLAlchemy	>=2.0.45, <2.1	ORM y acceso a datos con repositorios y unidades de trabajo.
Alembic	>=1.17, <2	Migraciones versionadas del esquema.
psycopg (binary)	>=3.3, <4	Controlador de PostgreSQL.
Pydantic Settings	>=2.11, <3	Configuración validada desde variables de entorno `FINANCE_AI_*`.
argon2-cffi	>=25.1, <26	Hash de contraseñas con Argon2.
email-validator	>=2.3, <3	Validación y normalización de correos.
limits	>=5, <6	Límites de tasa.
httpx	>=0.28, <1	Cliente HTTP hacia los proveedores de IA.
Pillow y pillow-heif	>=13, <14 · >=0.21	Validación y normalización de imágenes de recibos.
python-multipart	>=0.0.20, <1	Recepción de fotos en formularios multipart.

Servidor, herramientas de desarrollo: pytest 9, pytest-cov 7, Hypothesis 6, Ruff, mypy 1.18 y pip-audit.

Cliente (Dart y Flutter):

Tabla 12. *Dependencias del cliente.*

Nombre	Versión	Descripción
Flutter / Dart	3.47.2 / 3.13.2	Framework y lenguaje del cliente.
flutter_riverpod	3.4.2	Gestión de estado.
go_router	18.0.0	Navegación declarativa y rutas protegidas.
http	1.6.0	Cliente HTTP hacia la API.
flutter_secure_storage	11.0.0	Almacén seguro de la credencial renovable (Android y Windows).
camera e image_picker	^0.12.1 · ^1.2.3	Captura de fotos de recibos.
cryptography	^2.9.0	Cifrado de la foto confirmada del recibo en el dispositivo.
path_provider y shared_preferences	^2.1.6 · ^2.5.5	Rutas locales y preferencias (por ejemplo el tema).
lucide_icons_flutter	^3.1.25	Íconos de la interfaz.
flutter_lints	6.0.0	Reglas de estilo y análisis estático.

Para la voz, la aplicación integra reconocimiento local nativo en Android (con un motor sin conexión opcional descargable). El audio permanece en el dispositivo; el servidor solo recibe el texto.

5.2 ESTÁNDARES

Tabla 13. *Estándares y prácticas.*

Tema	Estándar o práctica aplicada
Metodología	CDIO (Concebir, Diseñar, Implementar, Operar) con ciclos de Scrum. El trabajo se organiza en historias de usuario (HU) y se refleja en un tablero de seguimiento.
Control de versiones	Git y GitHub. Cada historia se desarrolla en una rama propia a partir de `develop`; los cambios entran mediante *pull requests* y `main` recibe solo trabajo probado.

Tema	Estándar o práctica aplicada
Encriptación y secretos	Contraseñas con Argon2; tokens guardados como SHA-256; secretos solo en variables de entorno; almacenamiento seguro nativo en el cliente. En producción se recomienda TLS en el borde.
JWT	No se utiliza. Se eligieron tokens opacos respaldados por la base de datos porque permiten revocar sesiones de inmediato y rotar credenciales.
Correo	No aplica: el sistema no envía correos. El correo se usa solo como identificador de la cuenta.
Reportes	No se generan reportes en archivo. El panel de análisis se calcula con consultas SQL, sin IA.
Consumo de APIs	REST con JSON, rutas versionadas (`/api/v1`) y contrato OpenAPI 3.1. Las pasarelas a Gemini y DeepSeek usan la interfaz compatible con OpenAI, con reintento y respaldo entre proveedores.
Calidad del código	Ruff y mypy en el servidor, `flutter_lints` en el cliente, cobertura mínima del 80 % en el cliente y auditoría de dependencias.
Accesibilidad	El cliente se revisó contra WCAG 2.2 AA: semántica, foco visible, etiquetas, contraste y estados de carga.
Errores	Códigos estables, mensajes seguros e identificador de correlación en cada respuesta de error.
Configuración	Variables de entorno `FINANCE_AI_*` validadas al arrancar; plantilla `.env.example` sin secretos reales.

6 DIAGRAMAS DE CASOS DE USO

Los casos de uso se agrupan por funcionalidad. El actor principal es la **persona usuaria**, que se registra, inicia sesión y administra sus datos financieros. Las siguientes figuras muestran cada grupo.

6.1 REGISTRO E INICIO DE SESIÓN

Crear una cuenta, iniciar y cerrar sesión, y restaurar la sesión al reabrir la aplicación.

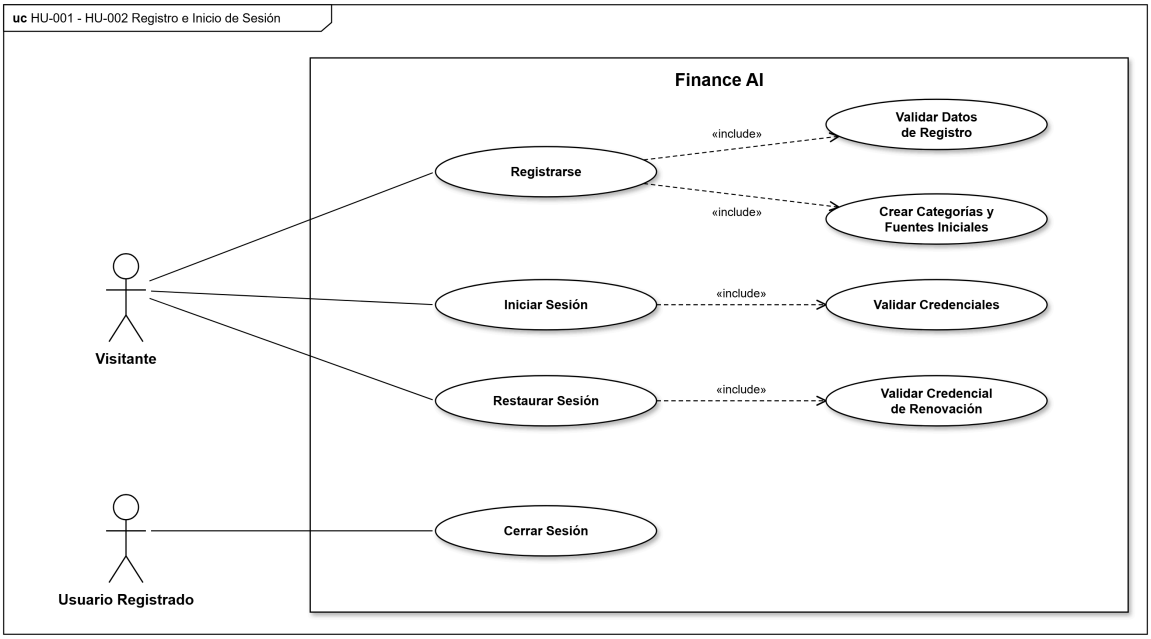


Figura 4. *Casos de uso: registro e inicio de sesión.*

6.2 CUENTAS DE DINERO

Crear, consultar, editar, desactivar, reactivar y eliminar cuentas, y transferir entre ellas.

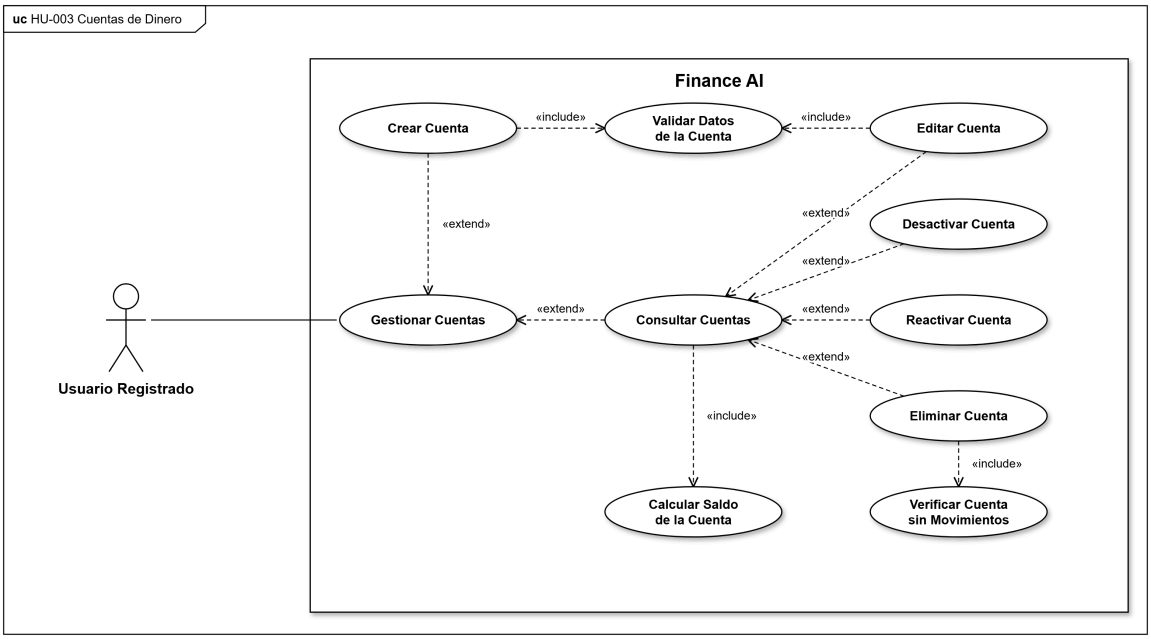


Figura 5. *Casos de uso: cuentas de dinero.*

6.3 REGISTRO DE MOVIMIENTOS

Registrar gastos e ingresos de forma manual o asistida por IA (texto, voz o foto).

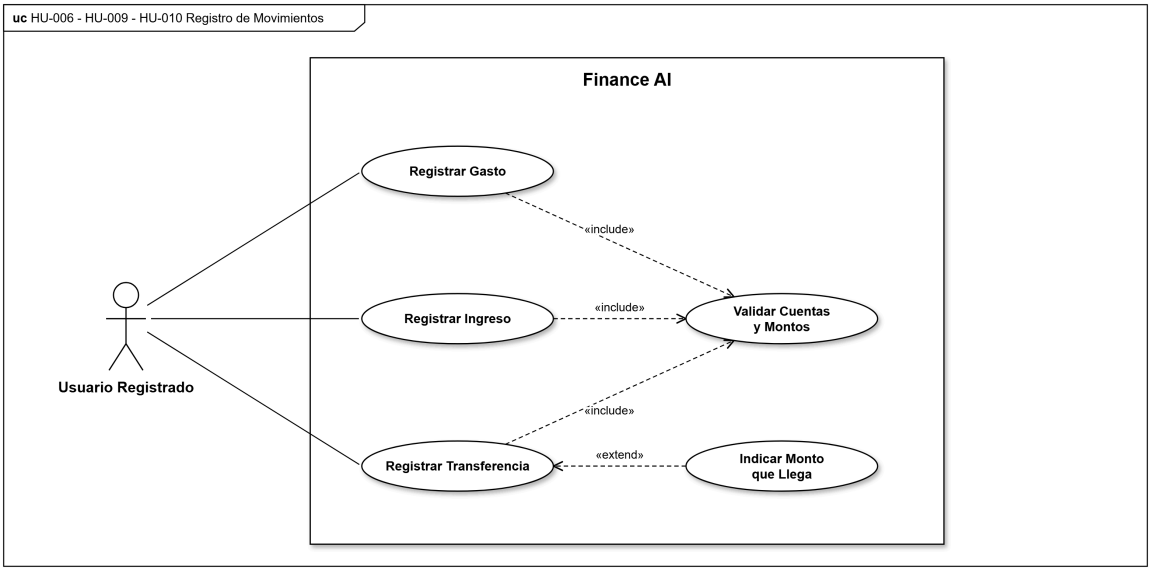


Figura 6. *Casos de uso: registro de movimientos.*

6.4 CORRECCIÓN DE GASTOS E INGRESOS

Consultar, corregir y eliminar movimientos ya confirmados.

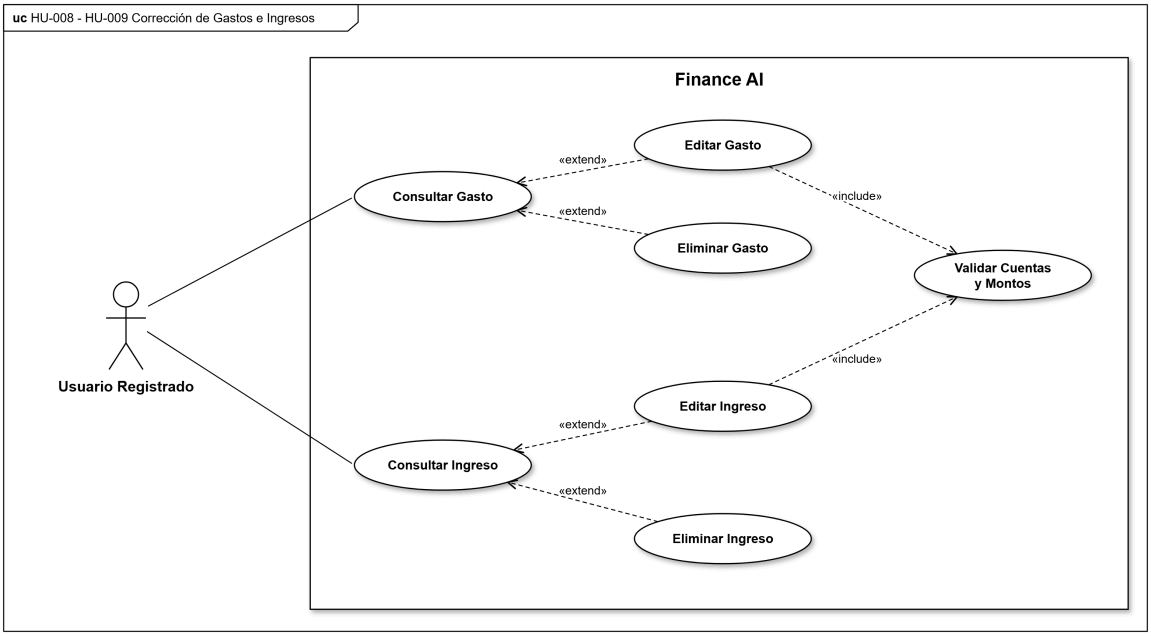


Figura 7. *Casos de uso: corrección de gastos e ingresos.*

6.5 HISTORIAL Y PANEL FINANCIERO

Buscar y filtrar el historial y consultar el resumen y el análisis de gastos.

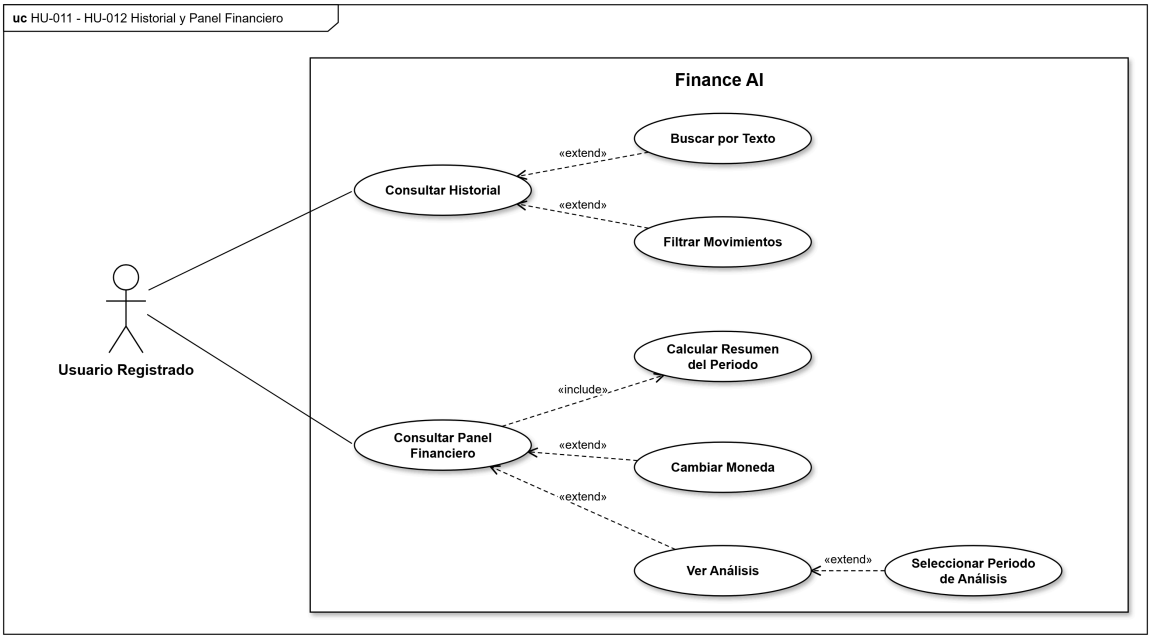


Figura 8. *Casos de uso: historial y panel financiero.*

6.6 CATEGORÍAS

Crear, editar, reordenar y eliminar categorías de gasto.

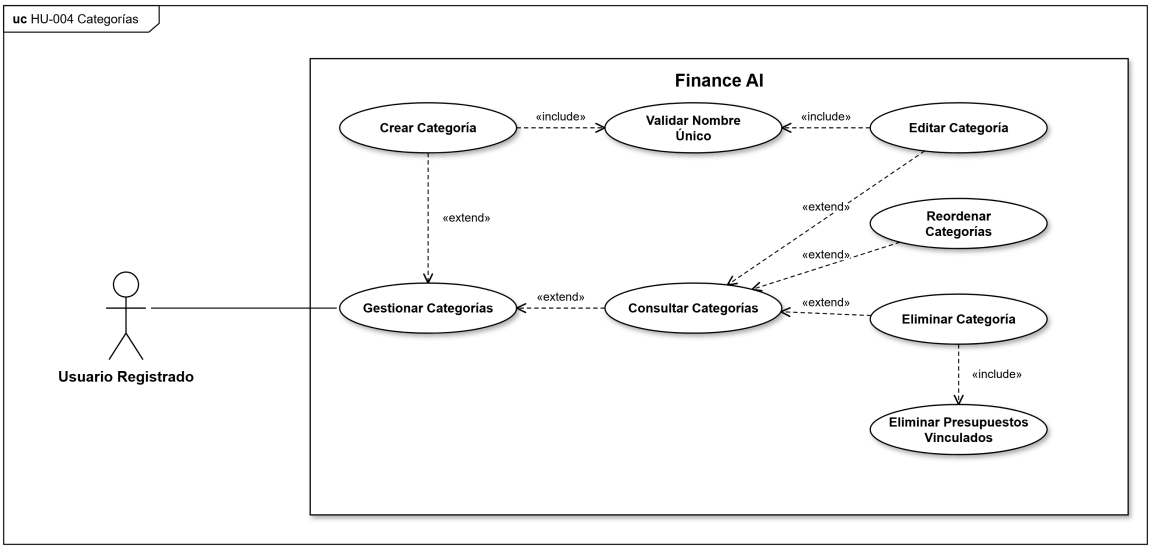


Figura 9. *Casos de uso: categorías.*

6.7 FUENTES DE INGRESO

Administrar las fuentes de ingreso de la persona.

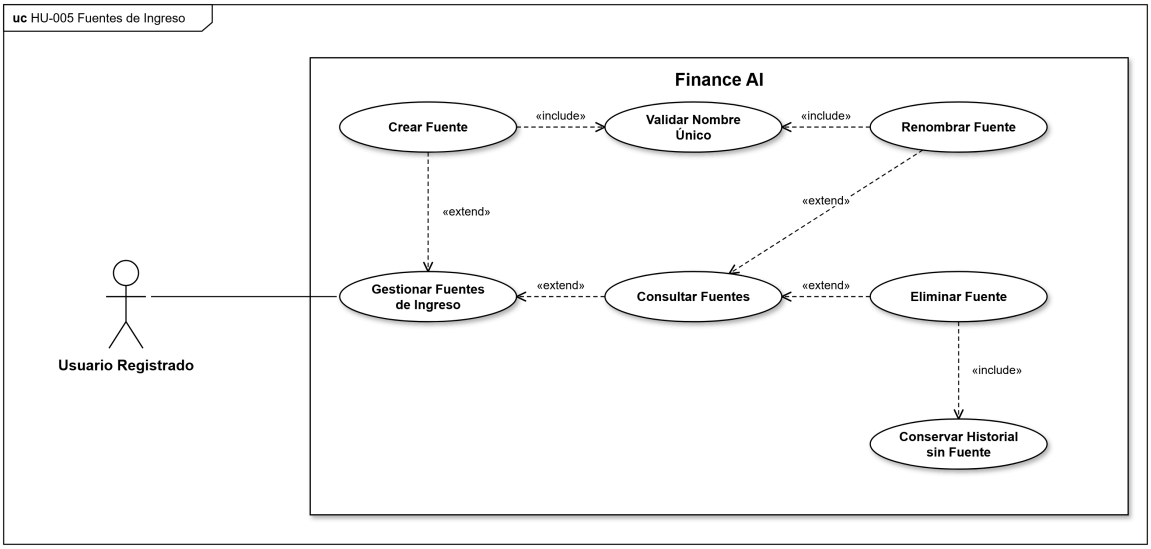


Figura 10. *Casos de uso: fuentes de ingreso.*

6.8 PRESUPUESTOS

Crear y administrar presupuestos y consultar su avance.

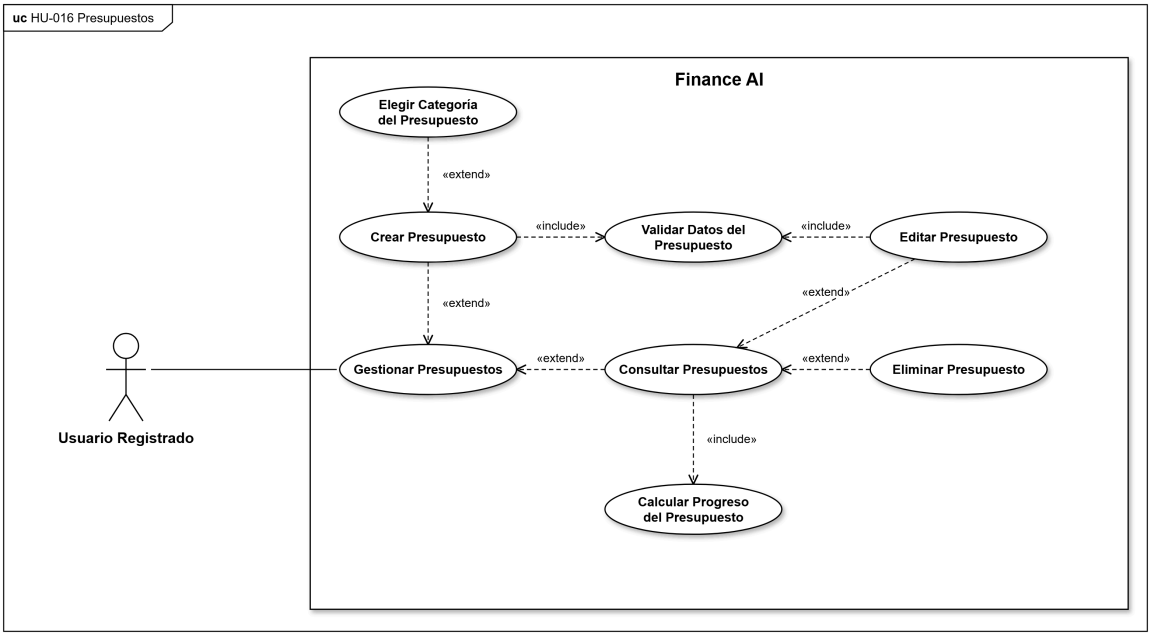


Figura 11. *Casos de uso: presupuestos.*

7 MODELO ENTIDAD RELACIÓN

El modelo tiene 21 tablas agrupadas en siete áreas. La figura general muestra las tablas y las relaciones principales; las figuras siguientes detallan cada área con sus columnas. En los diagramas, PK marca la clave primaria, FK la clave foránea y el asterisco marca las columnas obligatorias, es decir, las que no admiten valores nulos.

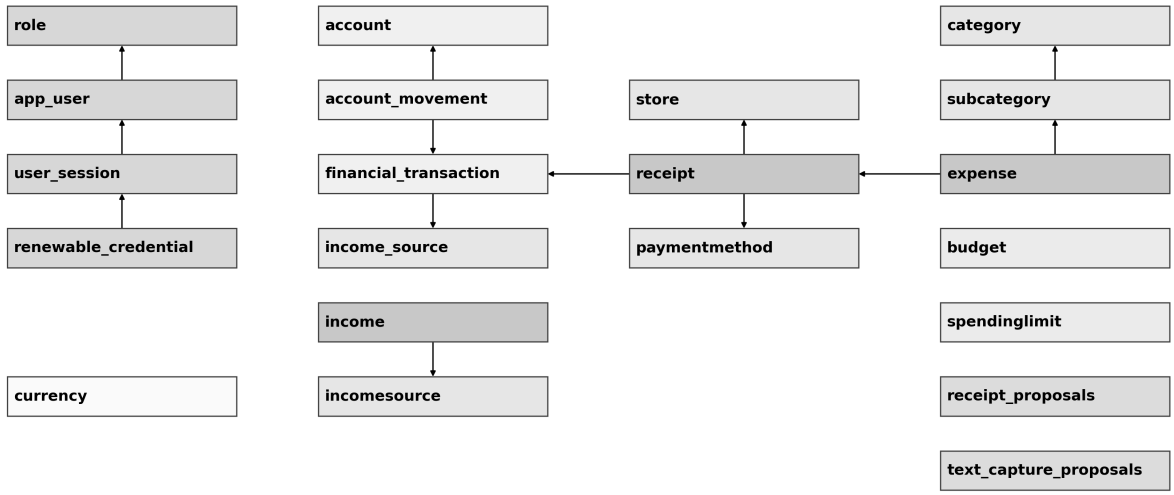


Figura 12. *Modelo entidad relación: vista general. Las claves a `app_user` (user_id) y a `currency` (currency_id) no se dibujan.*

7.1 IDENTIDAD Y SESIONES

Personas, roles, sesiones por dispositivo y credenciales renovables.

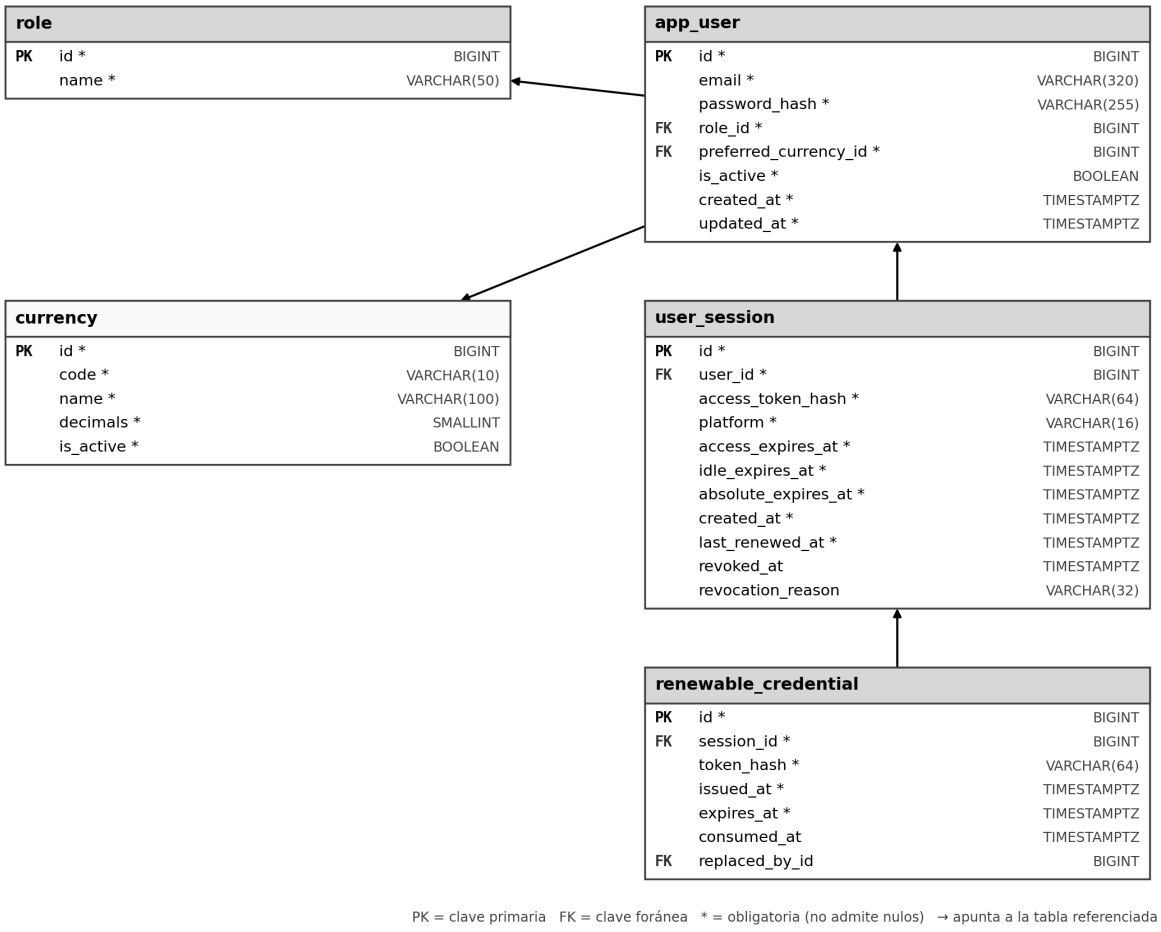


Figura 13. *Modelo entidad relación: identidad y sesiones. Un asterisco marca una columna obligatoria.*

7.2 CUENTAS Y TRANSACCIONES

Cuentas, transacciones financieras, movimientos de cuenta y las tablas de ingresos.

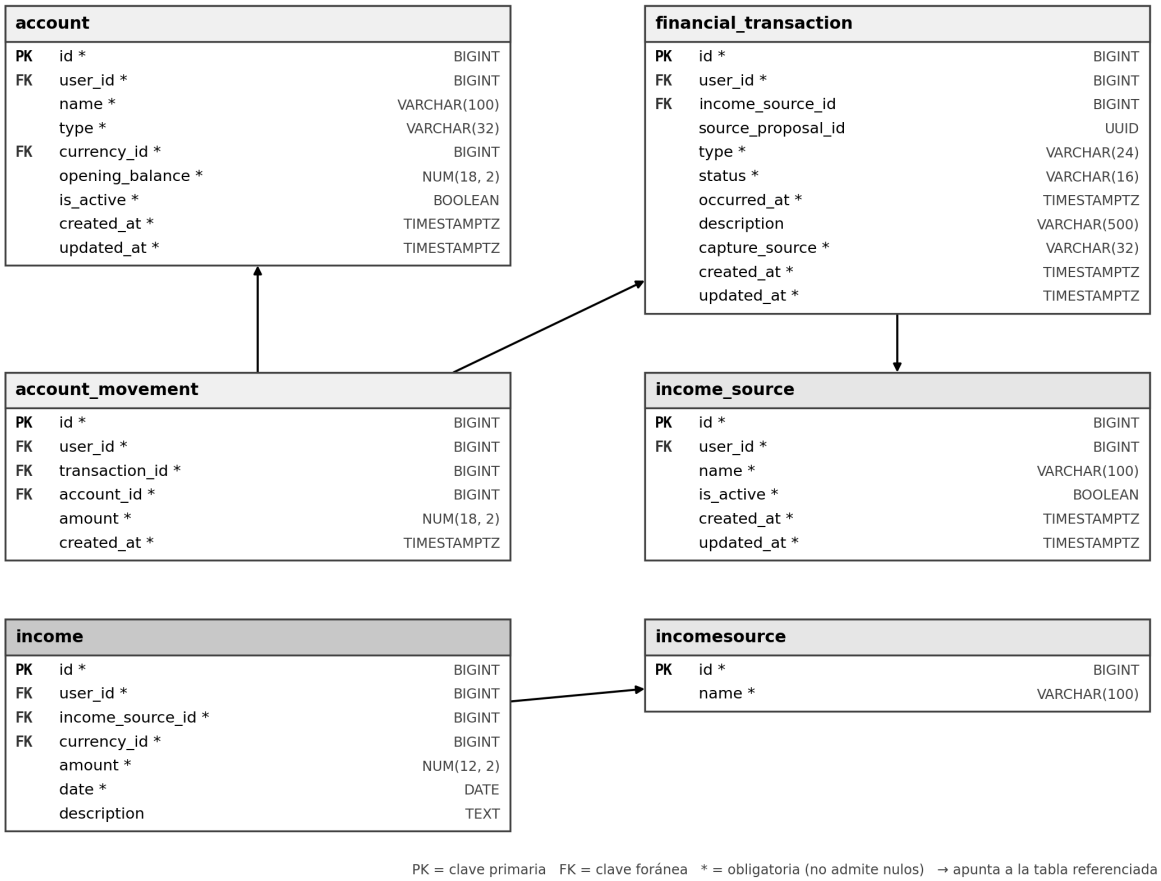


Figura 14. *Modelo entidad relación: cuentas y transacciones. Un asterisco marca una columna obligatoria.*

7.3 CATÁLOGOS Y RECIBOS

Categorías, subcategorías, comercios, métodos de pago y cabecera de recibos.

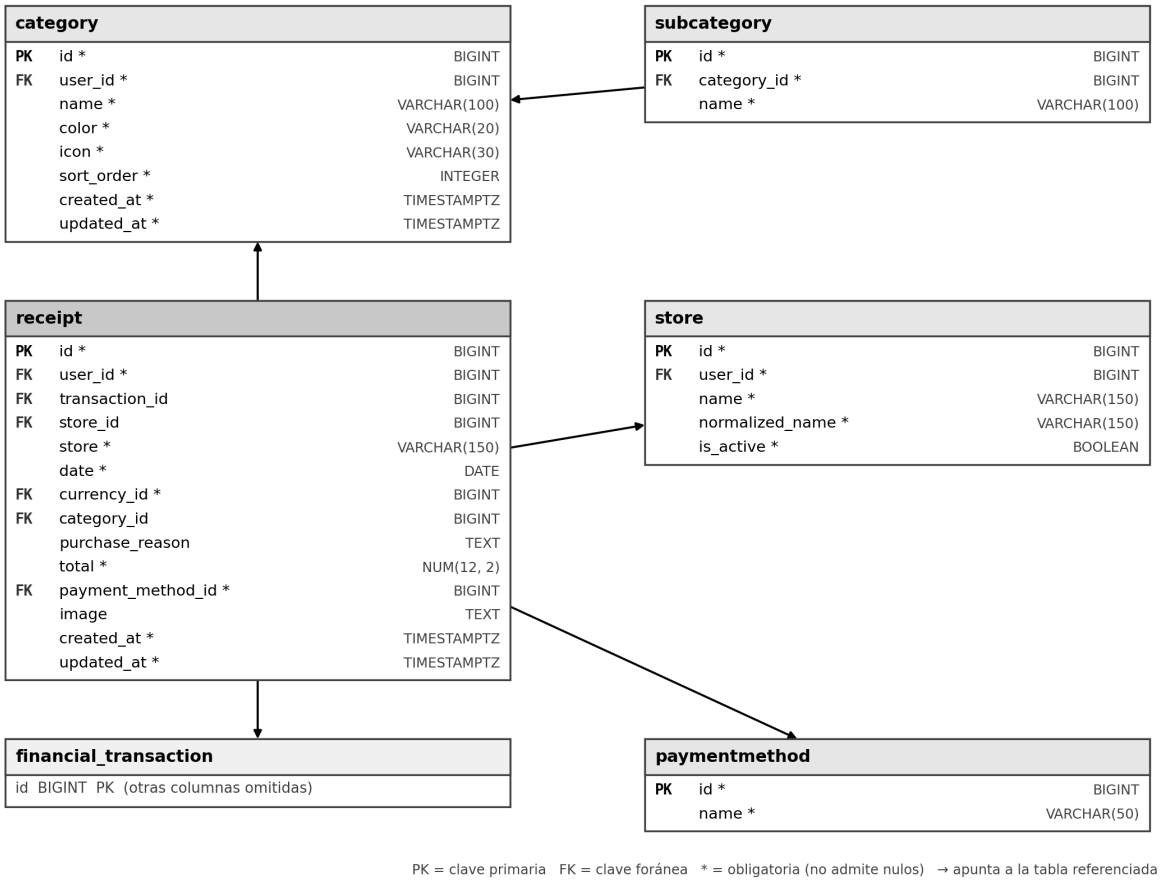


Figura 15. *Modelo entidad relación: catálogos y recibos. Un asterisco marca una columna obligatoria.*

7.4 GASTOS

Renglones de un recibo y sus categorías.

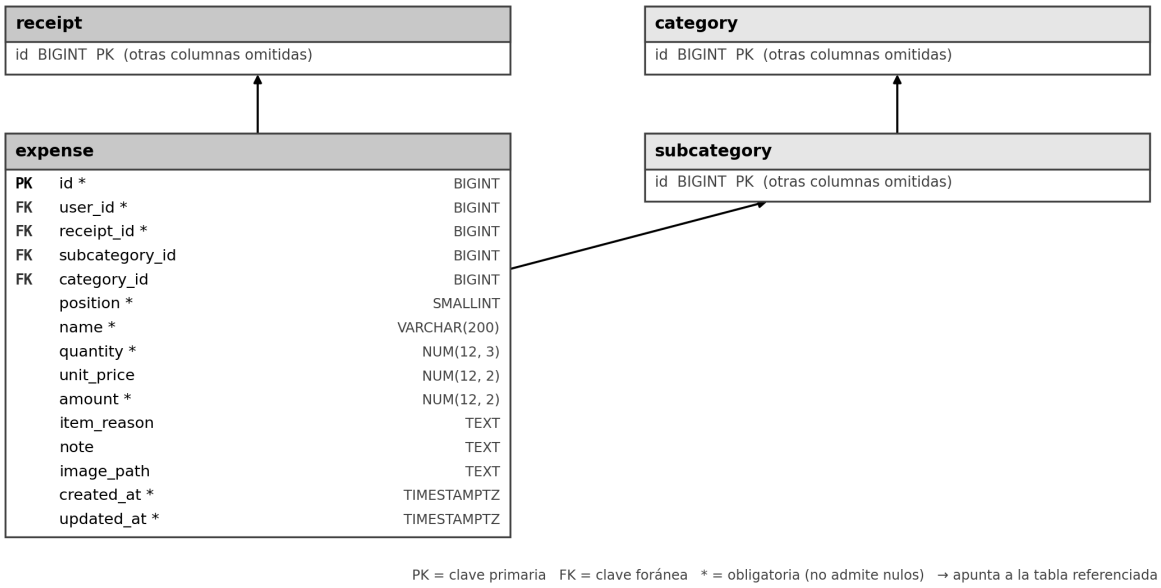


Figura 16. *Modelo entidad relación: gastos. Un asterisco marca una columna obligatoria.*

7.5 PRESUPUESTOS Y CAPTURAS CON IA

Presupuestos, límites de gasto y borradores de captura.

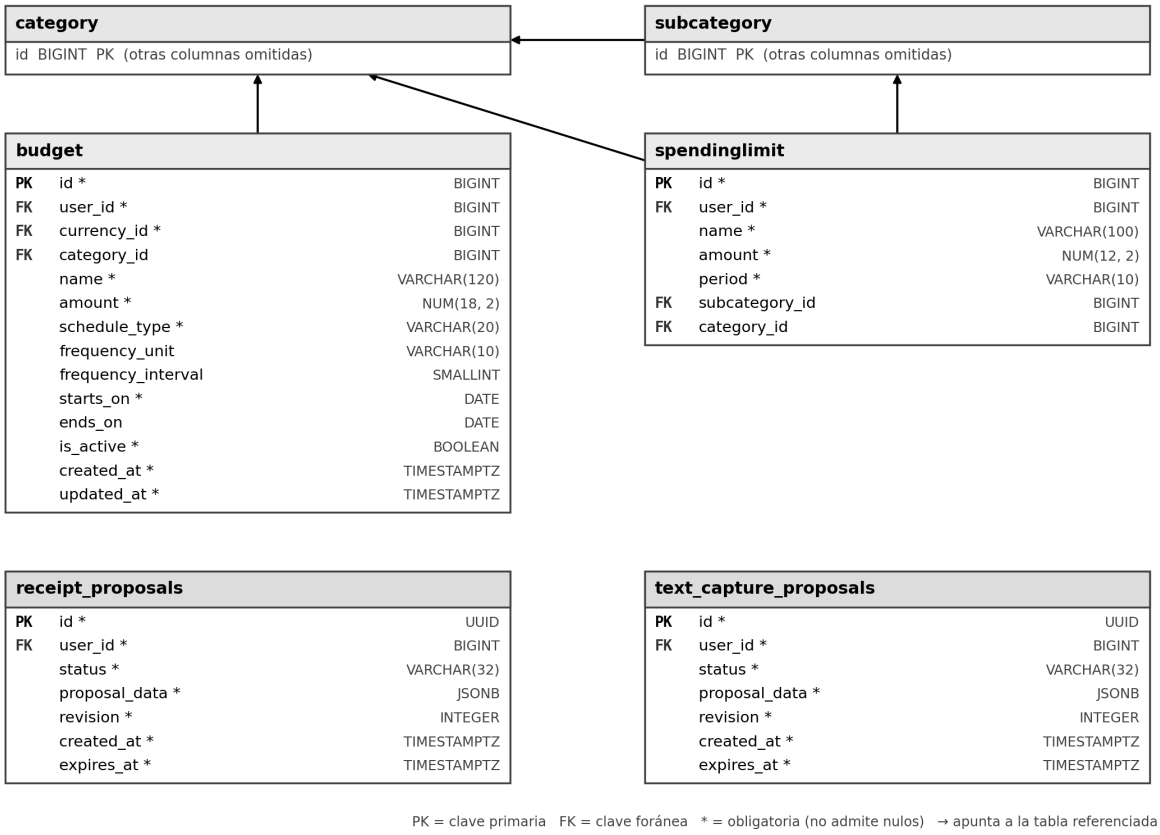


Figura 17. *Modelo entidad relación: presupuestos y capturas con ia. Un asterisco marca una columna obligatoria.*

Reglas de integridad relevantes: un movimiento de cuenta solo puede enlazar una cuenta y una transacción de la misma persona (clave foránea compuesta); un presupuesto solo puede apuntar a una categoría propia; borrar una transacción elimina en cascada su recibo, sus renglones y sus movimientos.

8 DICCIONARIO DE DATOS DEL SISTEMA

A continuación se presenta el diccionario completo de las 21 tablas y 156 columnas, con el formato del informe `Diccionario_Datos` de la plantilla de la asignatura. El mismo contenido se entrega como documento independiente, como hoja de cálculo y como consulta SQL.

8.1 IDENTIDAD Y SESIONES

8.1.1 Tabla role

Roles de la aplicación (por ejemplo `user` y `admin`). Cada usuario tiene exactamente un rol.

Tabla 14. *Columnas de la tabla `role`.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).
name	VARCHAR(50)	50	NO	NO		Nombre del rol (único).

8.1.2 Tabla app_user

Personas registradas. Guarda el correo normalizado, el hash de la contraseña y la moneda preferida.

Tabla 15. *Columnas de la tabla `app_user`.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).
email	VARCHAR(320)	320	NO	NO		Correo electrónico normalizado (minúsculas y sin espacios); único.
password_hash	VARCHAR(255)	255	NO	NO		Hash Argon2 de la contraseña; la contraseña nunca se almacena.
role_id	BIGINT	N/A	NO	NO	FK	Rol de la persona (clave foránea a `role`).
preferred_currency_id	BIGINT	N/A	NO	NO	FK	Moneda preferida de la persona (clave foránea a `currency`).
is_active	BOOLEAN	N/A	NO	NO		Indica si el registro está activo; al desactivarlo se conserva el historial.
created_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de creación.
updated_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de la última modificación.

8.1.3 Tabla user_session

Sesiones de acceso por dispositivo. Almacena solo el hash del token de acceso y sus tres vencimientos (acceso, inactividad y absoluto).

Tabla 16. *Columnas de la tabla `user_session`.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).
user_id	BIGINT	N/A	NO	NO	FK	Persona propietaria del registro (clave foránea a `app_user`); toda consulta filtra por este valor.
access_token_hash	VARCHAR(64)	64	NO	NO		Hash SHA-256 del token de acceso (64 caracteres hexadecimales); único.
platform	VARCHAR(16)	16	NO	NO		Plataforma del cliente que abrió la sesión: `android` o `windows`.
access_expires_at	TIMESTAMPTZ	N/A	NO	NO		Vencimiento del token de acceso (15 minutos por defecto).
idle_expires_at	TIMESTAMPTZ	N/A	NO	NO		Vencimiento por inactividad (30 días por defecto).
absolute_expires_at	TIMESTAMPTZ	N/A	NO	NO		Vencimiento absoluto de la sesión (90 días por defecto).
created_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de creación.
last_renewed_at	TIMESTAMPTZ	N/A	NO	NO		Última renovación de la sesión.

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
revoked_at	TIMESTAMPTZ	N/A	SÍ	NO		Fecha de revocación; nulo mientras la sesión siga vigente.
revocation_reason	VARCHAR(32)	32	SÍ	NO		Motivo de la revocación (por ejemplo `logout` o `session_limit`); obligatorio cuando existe `revoked_at`.

8.1.4 Tabla renewable_credential

Credenciales renovables de una sesión. Cada renovación consume la credencial vigente y emite otra (rotación); solo se guarda su hash.

Tabla 17. *Columnas de la tabla `renewable_credential`.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).
session_id	BIGINT	N/A	NO	NO	FK	Sesión a la que pertenece la credencial (clave foránea a `user_session`).
token_hash	VARCHAR(64)	64	NO	NO		Hash SHA-256 de la credencial renovable; único.
issued_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de emisión.
expires_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de vencimiento (posterior a la emisión).
consumed_at	TIMESTAMPTZ	N/A	SÍ	NO		Momento en que se usó para renovar; nulo si sigue vigente.
replaced_by_id	BIGINT	N/A	SÍ	NO	FK	Credencial que la reemplazó al rotar (autorreferencia).

8.2 REFERENCIA

8.2.1 Tabla currency

Monedas admitidas (código ISO 4217, nombre y decimales).

Tabla 18. *Columnas de la tabla `currency`.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).
code	VARCHAR(10)	10	NO	NO		Código ISO 4217 de la moneda (por ejemplo COP); único.
name	VARCHAR(100)	100	NO	NO		Nombre del registro.
decimals	SMALLINT	N/A	NO	NO		Número de decimales de la moneda.
is_active	BOOLEAN	N/A	NO	NO		Indica si el registro está activo; al desactivarlo se conserva el historial.

8.3 CUENTAS Y TRANSACCIONES

8.3.1 Tabla account

Cuentas donde la persona guarda su dinero (efectivo, banco, billetera digital, ahorros, tarjeta de crédito u otra). El saldo actual no se almacena: se calcula.

Tabla 19. *Columnas de la tabla `account`.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
user_id	BIGINT	N/A	NO	NO	FK	Persona propietaria del registro (clave foránea a `app_user`); toda consulta filtra por este valor.
name	VARCHAR(100)	100	NO	NO		Nombre de la cuenta; único entre las cuentas activas de la persona, sin distinguir mayúsculas.
type	VARCHAR(32)	32	NO	NO		Tipo de cuenta: `cash`, `bank`, `digital_wallet`, `savings`, `credit_card` u `other`.
currency_id	BIGINT	N/A	NO	NO	FK	Moneda del registro (clave foránea a `currency`).
opening_balance	NUMERIC(18, 2)	N/A	NO	NO		Saldo inicial con signo (negativo representa deuda). El saldo actual se calcula.
is_active	BOOLEAN	N/A	NO	NO		Falso cuando la cuenta fue desactivada; conserva su historial.
created_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de creación.
updated_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de la última modificación.

8.3.2 Tabla financial_transaction

Transacción financiera confirmada, pendiente o anulada (gasto, ingreso, transferencia o ajuste). Es la cabecera común de todo movimiento de dinero.

Tabla 20. *Columnas de la tabla `financial_transaction`.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).
user_id	BIGINT	N/A	NO	NO	FK	Persona propietaria del registro (clave foránea a `app_user`); toda consulta filtra por este valor.
income_source_id	BIGINT	N/A	SÍ	NO	FK	Fuente de ingreso (solo para transacciones de tipo `income`).
source_proposal_id	UUID	N/A	SÍ	NO		Borrador de IA del que proviene la transacción; con `user_id` es único y hace idempotente la confirmación.
type	VARCHAR(24)	24	NO	NO		Tipo: `expense`, `income`, `transfer` o `adjustment`.
status	VARCHAR(16)	16	NO	NO		Estado: `pending`, `confirmed` o `voided`. Solo las confirmadas afectan los saldos.
occurred_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora en que ocurrió el movimiento.
description	VARCHAR(500)	500	SÍ	NO		Descripción libre del movimiento.
capture_source	VARCHAR(32)	32	NO	NO		Origen del registro: `manual`, `ai`, `text_ai`, `audio_ai`, `receipt_photo_ai` o `import`.
created_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de creación.
updated_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de la última modificación.

8.3.3 Tabla account_movement

Efecto firmado de una transacción sobre una cuenta. El saldo actual de una cuenta es su saldo inicial más la suma de estos movimientos confirmados.

Tabla 21. *Columnas de la tabla `account_movement`.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
user_id	BIGINT	N/A	NO	NO	FK	Propietario; junto con la cuenta y la transacción garantiza que ambas pertenezcan a la misma persona.
transaction_id	BIGINT	N/A	NO	NO	FK	Transacción que originó el movimiento (clave foránea a `financial_transaction`).
account_id	BIGINT	N/A	NO	NO	FK	Cuenta afectada (clave foránea a `account`).
amount	NUMERIC(18, 2)	N/A	NO	NO		Monto con signo (positivo acredita, negativo debita); no puede ser cero.
created_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de creación.

8.4 CATÁLOGOS

8.4.1 Tabla category

Categorías de gasto definidas por cada persona, con color, ícono y orden de presentación.

Tabla 22. *Columnas de la tabla `category`.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).
user_id	BIGINT	N/A	NO	NO	FK	Persona propietaria del registro (clave foránea a `app_user`); toda consulta filtra por este valor.
name	VARCHAR(100)	100	NO	NO		Nombre de la categoría, sin espacios en los extremos.
color	VARCHAR(20)	20	NO	NO		Color de la etiqueta: azul, verde, naranja, morado, rosa, rojo, turquesa, amarillo o gris (en inglés en la base).
icon	VARCHAR(30)	30	NO	NO		Identificador del ícono, dentro de un conjunto cerrado (alimentación, transporte, hogar, salud, etc.).
sort_order	INTEGER	N/A	NO	NO		Posición de la categoría en la lista; no negativa.
created_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de creación.
updated_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de la última modificación.

8.4.2 Tabla subcategory

Subcategorías opcionales dentro de una categoría.

Tabla 23. *Columnas de la tabla `subcategory`.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).
category_id	BIGINT	N/A	NO	NO	FK	Categoría a la que pertenece (clave foránea a `category`).
name	VARCHAR(100)	100	NO	NO		Nombre de la subcategoría.

8.4.3 Tabla paymentmethod

Catálogo de métodos de pago asociados a los recibos (efectivo, tarjeta, etc.).

Tabla 24. *Columnas de la tabla `paymentmethod`.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).
name	VARCHAR(50)	50	NO	NO		Nombre del método de pago; único.

8.4.4 Tabla income_source

Fuentes de ingreso propias de cada persona (por ejemplo salario). Las usan las transacciones de tipo ingreso.

Tabla 25. *Columnas de la tabla* `income_source`*.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).
user_id	BIGINT	N/A	NO	NO	FK	Persona propietaria del registro (clave foránea a `app_user`); toda consulta filtra por este valor.
name	VARCHAR(100)	100	NO	NO		Nombre de la fuente de ingreso.
is_active	BOOLEAN	N/A	NO	NO		Indica si el registro está activo; al desactivarlo se conserva el historial.
created_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de creación.
updated_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de la última modificación.

8.4.5 Tabla incomesource

Catálogo global de fuentes de ingreso del diseño inicial; lo usa la tabla `income`. Se conserva por compatibilidad: los ingresos actuales usan `income_source`.

Tabla 26. *Columnas de la tabla* `incomesource`*.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).
name	VARCHAR(100)	100	NO	NO		Nombre de la fuente global; único.

8.4.6 Tabla store

Comercios normalizados por persona, para reutilizarlos entre recibos.

Tabla 27. *Columnas de la tabla* `store`*.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).
user_id	BIGINT	N/A	NO	NO	FK	Persona propietaria del registro (clave foránea a `app_user`); toda consulta filtra por este valor.
name	VARCHAR(150)	150	NO	NO		Nombre del comercio tal como se registró.
normalized_name	VARCHAR(150)	150	NO	NO		Nombre normalizado para detectar duplicados; único por persona.
is_active	BOOLEAN	N/A	NO	NO		Indica si el registro está activo; al desactivarlo se conserva el historial.

8.5 RECIBOS, GASTOS E INGRESOS

8.5.1 Tabla receipt

Cabecera de un recibo o gasto manual: comercio, fecha, moneda, total, método de pago y vínculo con la transacción financiera.

Tabla 28. *Columnas de la tabla* `receipt`*.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
user_id	BIGINT	N/A	NO	NO	FK	Persona propietaria del registro (clave foránea a `app_user`); toda consulta filtra por este valor.
transaction_id	BIGINT	N/A	SÍ	NO	FK	Transacción financiera asociada (clave foránea a `financial_transaction`).
store_id	BIGINT	N/A	SÍ	NO	FK	Comercio normalizado (clave foránea a `store`); opcional.
store	VARCHAR(150)	150	NO	NO		Nombre del comercio mostrado en el recibo.
date	DATE	N/A	NO	NO		Fecha del comprobante.
currency_id	BIGINT	N/A	NO	NO	FK	Moneda del registro (clave foránea a `currency`).
category_id	BIGINT	N/A	SÍ	NO	FK	Categoría general del recibo; opcional.
purchase_reason	TEXT	N/A	SÍ	NO		Motivo de la compra (texto libre).
total	NUMERIC(12, 2)	N/A	NO	NO		Total del recibo; mayor que cero.
payment_method_id	BIGINT	N/A	NO	NO	FK	Método de pago (clave foránea a `paymentmethod`).
image	TEXT	N/A	SÍ	NO		Referencia a la imagen adjunta; el servidor no conserva el archivo.
created_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de creación.
updated_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de la última modificación.

8.5.2 Tabla expense

Renglones (ítems) de un recibo o gasto: nombre, cantidad, precio unitario y monto.

Tabla 29. *Columnas de la tabla* `expense`*.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).
user_id	BIGINT	N/A	NO	NO	FK	Persona propietaria del registro (clave foránea a `app_user`); toda consulta filtra por este valor.
receipt_id	BIGINT	N/A	NO	NO	FK	Recibo al que pertenece el renglón (clave foránea a `receipt`).
subcategory_id	BIGINT	N/A	SÍ	NO	FK	Subcategoría del renglón; opcional.
category_id	BIGINT	N/A	SÍ	NO	FK	Categoría del renglón; opcional.
position	SMALLINT	N/A	NO	NO		Posición del renglón dentro del recibo; única por recibo.
name	VARCHAR(200)	200	NO	NO		Descripción del ítem o concepto.
quantity	NUMERIC(12, 3)	N/A	NO	NO		Cantidad (hasta tres decimales); mayor que cero.
unit_price	NUMERIC(12, 2)	N/A	SÍ	NO		Precio unitario; opcional.
amount	NUMERIC(12, 2)	N/A	NO	NO		Monto del renglón; mayor que cero.
item_reason	TEXT	N/A	SÍ	NO		Motivo del ítem (texto libre).
note	TEXT	N/A	SÍ	NO		Nota adicional.
image_path	TEXT	N/A	SÍ	NO		Referencia a una imagen adjunta al renglón.
created_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de creación.
updated_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de la última modificación.

8.5.3 Tabla income

Ingresos del diseño inicial (monto, fecha y fuente global). Se conserva por compatibilidad: la aplicación actual registra los ingresos como transacciones financieras de tipo `income`.

Tabla 30. *Columnas de la tabla* `income`*.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).
user_id	BIGINT	N/A	NO	NO	FK	Persona propietaria del registro (clave foránea a `app_user`); toda consulta filtra por este valor.
income_source_id	BIGINT	N/A	NO	NO	FK	Fuente global del ingreso (clave foránea a `incomesource`).
currency_id	BIGINT	N/A	NO	NO	FK	Moneda del registro (clave foránea a `currency`).
amount	NUMERIC(12, 2)	N/A	NO	NO		Monto del ingreso; mayor que cero.
date	DATE	N/A	NO	NO		Fecha del ingreso.
description	TEXT	N/A	SÍ	NO		Descripción libre.

8.6 PRESUPUESTOS

8.6.1 Tabla budget

Presupuestos de gasto: alcance (todos los gastos o una categoría), calendario recurrente o de una sola vez, monto y moneda. El avance se calcula al consultar.

Tabla 31. *Columnas de la tabla* `budget`*.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).
user_id	BIGINT	N/A	NO	NO	FK	Propietario; junto con la categoría garantiza que la categoría sea de la misma persona.
currency_id	BIGINT	N/A	NO	NO	FK	Moneda del registro (clave foránea a `currency`).
category_id	BIGINT	N/A	SÍ	NO	FK	Categoría que limita el presupuesto; nulo significa todos los gastos.
name	VARCHAR(120)	120	NO	NO		Nombre del presupuesto (opcional en la aplicación).
amount	NUMERIC(18, 2)	N/A	NO	NO		Monto objetivo; mayor que cero.
schedule_type	VARCHAR(20)	20	NO	NO		Calendario: `recurring` (recurrente) o `one_time` (una sola vez).
frequency_unit	VARCHAR(10)	10	SÍ	NO		Unidad de repetición: `day`, `week`, `month` o `year`; solo si es recurrente.
frequency_interval	SMALLINT	N/A	SÍ	NO		Cada cuántas unidades se repite (1 a 999); solo si es recurrente.
starts_on	DATE	N/A	NO	NO		Fecha de inicio.
ends_on	DATE	N/A	SÍ	NO		Fecha de fin; nulo significa sin fin. No puede ser anterior al inicio.
is_active	BOOLEAN	N/A	NO	NO		Indica si el registro está activo; al desactivarlo se conserva el historial.
created_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de creación.
updated_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de la última modificación.

8.6.2 Tabla spendinglimit

Límites de gasto semanales o mensuales del diseño inicial. La aplicación actual gestiona sus presupuestos en `budget`; la tabla se conserva por compatibilidad.

Tabla 32. *Columnas de la tabla* `spendinglimit`*.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	BIGINT	N/A	NO	SÍ	PK	Identificador interno de la fila (clave primaria).
user_id	BIGINT	N/A	NO	NO	FK	Persona propietaria del registro (clave foránea a `app_user`); toda consulta filtra por este valor.
name	VARCHAR(100)	100	NO	NO		Nombre del límite.
amount	NUMERIC(12, 2)	N/A	NO	NO		Monto máximo; mayor que cero.
period	VARCHAR(10)	10	NO	NO		Período: `weekly` o `monthly`.
subcategory_id	BIGINT	N/A	SÍ	NO	FK	Subcategoría limitada; exactamente una de subcategoría o categoría.
category_id	BIGINT	N/A	SÍ	NO	FK	Categoría limitada; exactamente una de subcategoría o categoría.

8.7 CAPTURAS CON IA

8.7.1 Tabla receipt_proposals

Borradores de captura de recibos con IA (JSON de la propuesta, revisión y vencimiento). Nada se guarda como gasto hasta que la persona confirma.

Tabla 33. *Columnas de la tabla* `receipt_proposals`*.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	UUID	N/A	NO	NO	PK	Identificador interno de la fila (clave primaria).
user_id	BIGINT	N/A	NO	NO	FK	Persona propietaria del registro (clave foránea a `app_user`); toda consulta filtra por este valor.
status	VARCHAR(32)	32	NO	NO		Estado del borrador: `complete`, `partial`, `not_a_receipt`, `failed_extraction`, `confirmed` o `discarded`.
proposal_data	JSONB	N/A	NO	NO		Contenido de la propuesta en formato JSON.
revision	INTEGER	N/A	NO	NO		Número de revisión del borrador.
created_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de creación.
expires_at	TIMESTAMPTZ	N/A	NO	NO		Vencimiento del borrador.

8.7.2 Tabla text_capture_proposals

Borradores de captura por texto o voz con IA. Vencen a las 24 horas y solo los ve su dueño.

Tabla 34. *Columnas de la tabla* `text_capture_proposals`*.*

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
id	UUID	N/A	NO	NO	PK	Identificador interno de la fila (clave primaria).
user_id	BIGINT	N/A	NO	NO	FK	Persona propietaria del registro (clave foránea a `app_user`); toda consulta filtra por este valor.
status	VARCHAR(32)	32	NO	NO		Estado del borrador (por ejemplo `ready`, `needs_clarification`, `unrecognized`, `confirmed` o `discarded`).
proposal_data	JSONB	N/A	NO	NO		Contenido de la propuesta en formato JSON.
revision	INTEGER	N/A	NO	NO		Número de revisión del borrador.

Columna	Tipo de dato	Long.	Nulo	Ident.	Clave	Descripción
created_at	TIMESTAMPTZ	N/A	NO	NO		Fecha y hora de creación.
expires_at	TIMESTAMPTZ	N/A	NO	NO		Vencimiento del borrador (24 horas).

9 ANEXOS

9.1 PROCEDIMIENTOS ALMACENADOS Y DISPARADORES

Finance AI **no utiliza procedimientos almacenados ni disparadores (*triggers*) propios**. La lógica de negocio vive en la capa de servicios del servidor, que ejecuta cada operación dentro de una transacción atómica, y la integridad de los datos la garantizan restricciones declarativas de PostgreSQL. Las dos tablas siguientes documentan los mecanismos que cumplen el papel de esas piezas.

9.1.1 Mecanismos de la base de datos equivalentes a disparadores

Tabla 35. *Mecanismos de integridad en la base de datos.*

N.º	Mecanismo	Dónde	Qué garantiza
1	Clave foránea compuesta	`account_movement (user_id, account_id)` y `(user_id, transaction_id)`	Un movimiento nunca mezcla cuentas y transacciones de personas distintas.
2	Borrado en cascada	`receipt`, `expense` y `account_movement` respecto de `financial_transaction`	Eliminar un gasto borra de forma atómica su recibo, sus renglones y sus movimientos.
3	Restricción CHECK de monto	`account_movement.amount <> 0`, `expense.amount > 0`, `budget.amount > 0`	No se aceptan movimientos nulos ni montos inválidos.
4	Clave única de idempotencia	`financial_transaction (user_id, source_proposal_id)`	Confirmar dos veces un borrador de IA devuelve el mismo resultado y no duplica el movimiento.
5	CHECK de coherencia del calendario	`budget` (tipo, unidad e intervalo)	Un presupuesto recurrente exige unidad e intervalo; uno de una sola vez no los admite.
6	CHECK de vigencias	`user_session` y `renewable_credential`	Los vencimientos de acceso, inactividad y absoluto respetan su orden; una credencial reemplazada ya está consumida.
7	Índice único parcial por nombre	`account` (nombre sin distinguir mayúsculas entre cuentas activas)	No hay cuentas activas duplicadas por persona.

9.1.2 Operaciones atómicas del servidor equivalentes a procedimientos almacenados

Tabla 36. *Operaciones transaccionales del servidor.*

N.º	Operación	Servicio	Qué hace en una sola transacción
1	Registrar una persona	`auth`	Crea el usuario, su rol, su moneda y los datos iniciales (categorías y fuentes por defecto).
2	Iniciar y renovar sesión	`auth`	Emite el token y la credencial, consume la anterior (rotación) y respeta el límite de sesiones activas.
3	Registrar un gasto	`expenses`	Crea la transacción, el recibo, los renglones y el movimiento firmado de la cuenta.
4	Transferir entre cuentas	`transfers`	Crea una transferencia con dos movimientos opuestos y evita conflictos de concurrencia.

N.º	Operación	Servicio	Qué hace en una sola transacción
5	Corregir un gasto	`expenses`	Recalcula el total del recibo y el movimiento de cuenta dentro de la misma transacción.
6	Confirmar una captura con IA	`capture`, `receipts`	Valida el borrador, registra el movimiento por el mismo servicio manual y lo marca como confirmado de forma idempotente.
7	Calcular el avance de un presupuesto	`budgets`	Suma solo gastos confirmados de la ocurrencia vigente y de la misma moneda; no almacena el total.

También existe una vista de compatibilidad (`finance_ai_view.category`) creada por la migración `0006_category_management`.

9.2 EVIDENCIAS DE EJECUCIÓN

9.2.1 Migraciones de la base de datos

Tabla 37. *Migraciones de Alembic de la rama `main` (la cabeza es `0017_text_capture_idempotency`).*

Revisión	Archivo de migración
0001	`0001_uow01_foundation_registration`
0002	`0002_uow02_secure_sessions`
0003	`0003_uow02_constraint_names`
0004	`0004_uow03_financial_accounts`
0005	`0005_drop_payment_method`
0006	`0006_category_management`
0007	`0007_uow03_income_source`
0008	`0008_complete_dbml_schema`
0009	`0009_hu008_hu011_expense_history`
0010	`0010_expense_audit_index`
0011	`0011_hu009_income_source_index`
0012	`0012_dbml_store_budget`
0013	`0013_align_indexes_with_models`
0014	`0014_hg016_budgets`
0014	`0014_receipt_proposals_table`
0015	`0015_merge_hu015_budgets`
0016	`0016_hu013_text_capture_proposals`
0017	`0017_text_capture_financial_idempotency`

Las revisiones se aplican en orden con `alembic upgrade head`; la integración verifica la cadena completa (bajar a la base y subir a la cabeza) sobre una base `_test`.

9.2.2 Servicios en ejecución y comprobación de salud

Estado de los contenedores del servidor durante las pruebas del 11 de noviembre de 2026 y respuesta de la comprobación de salud:

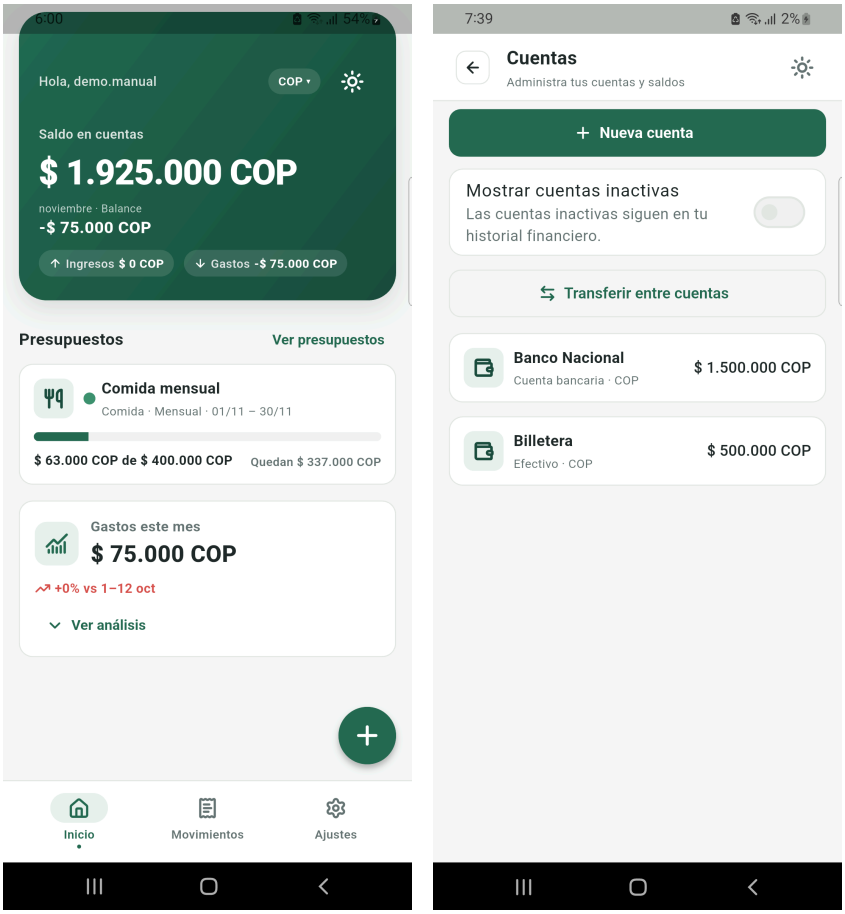
Tabla 38. *Evidencia de servicios en ejecución.*

Servicio	Estado
`finance-ai-api-1`	Up, healthy

Servicio	Estado
`finance-ai-database-1`	Up, healthy
`GET /health/ready`	`{"status":"ready"}`

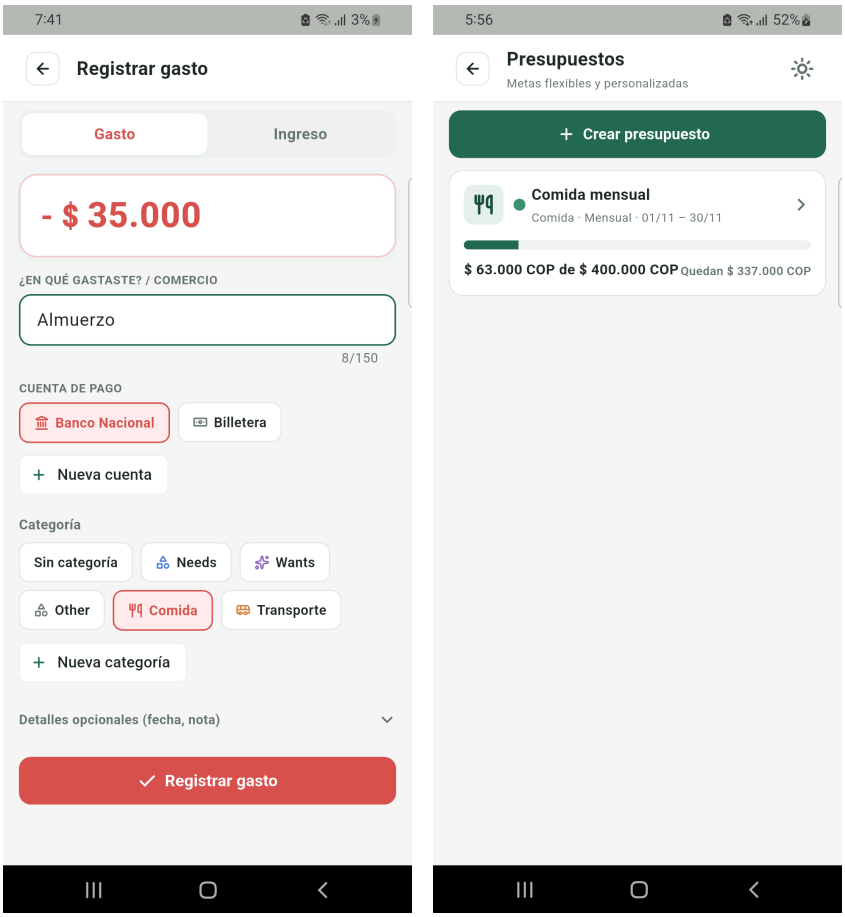
9.2.3 Funcionamiento del cliente

Las siguientes capturas, tomadas en un teléfono Android conectado al servidor local con una cuenta de prueba, muestran flujos completos: resumen, cuentas, registro de un gasto, historial y presupuestos.



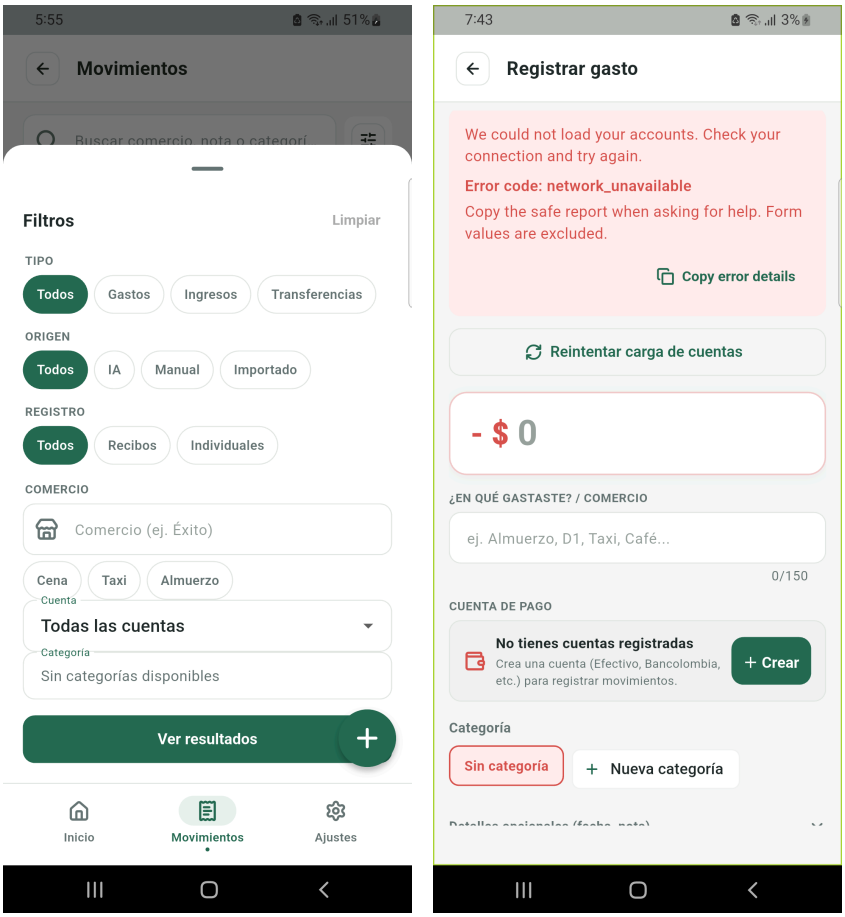
Inicio con saldo y presupuesto *Cuentas y saldos*

Figura 18. *Pantallas de Inicio y Cuentas (datos de prueba).*



Formulario de gasto *Presupuestos con avance*

Figura 19. *Registro de un gasto y avance de un presupuesto.*



Filtros del historial *Error con detalles copiables*

Figura 20. *Filtros del historial y manejo de errores.*